Testing Policy

Last Revision: 26/06/2026

David van Rooijen

Dandré Nel

Daniel Cohen

Stephan Kritzinger

Michael Koch

TABLE OF CONTENTS

Introduction

Savanna Sentinel is a wildlife conservation monitoring PWA used by rangers, analysts, community liaisons, and administrators who frequently operate in areas with unreliable or no connectivity. The system handles sensitive data: patrol routes, poaching incidents, and informant tip-offs, which makes security and correctness central concerns rather than optional considerations.

This document outlines the testing policy followed during development. It covers the testing philosophy, the levels of testing applied, the strategies used in execution, and the outcomes of those efforts. Each section describes the rationale for the approach taken, how it was implemented, and why it matters to the system's success.

Testing Strategy Overview

The testing strategy followed a layered approach, aligned with industry best practices:

- Unit Testing was applied to verify the correctness of individual service functions and repository methods in isolation.
- Frontend Component Testing validated UI components in isolation.
- Integration Testing validated that modules interact correctly with one another, particularly around authentication, role enforcement, and data persistence.
- End-to-End (E2E) Testing will simulate real-world user scenarios, validating complete workflows from login through to task completion. Tool: Cypress.

Catching defects at lower levels is significantly cheaper than finding them at integration or production stage. The strategy is front-loaded to reflect this.

1. Unit Testing

Overview

Unit testing targeted the smallest testable components of the backend - service functions, repository methods, and JWT utilities. By testing in isolation with mocked dependencies, each function's behaviour was confirmed under normal conditions, error conditions, and edge cases without relying on a running database or external services.

Approach

Tests are implemented using pytest. External dependencies such as the database and email relay are replaced with in-memory fakes or mocks. Each function was tested under:

- Normal conditions (valid input producing expected output).
- Error conditions (missing fields, wrong credentials, non-existent records).
- Edge cases (duplicate entries, expired tokens, already-revoked tokens).

Relevance

Unit tests catch logic errors before they propagate into integration. Several of the bugs found during this phase, such as missing token revocation on password change and incorrect empty field validation would have been significantly harder to diagnose at a higher level. Correctness at this layer directly reduces integration failures later.

Coverage across modules

Auth module

- login - handles valid credentials, wrong password, unknown username, and inactive accoun.
- refresh - handles valid token exchange, token rotation, invalid tokens, and access tokens used as refresh tokens.
- logout - token revocation and silent handling of already-revoked tokens.
- register - handles valid registration, duplicate email, duplicate username, and short password rejection.

JWT Module

- encode / decode - valid round-trip, invalid data types, malformed tokens.
- Token expiry - access token (60 min) and refresh token (30 days) expiry.
- generate_token_pair - returns correct token structure and JTI, missing id raises error.
- get_current_user - invalid token, missing user ID, deactivated user, and success cases.
- require_admin - allows admin role, rejects all others.

User Module

- get_users - pagination and filtering by is_active and role.
- get_me - returns current user.
- update_me - name updates, password change with validation, token revocation on password change.
- change_role - success and not found cases.
- switch_status - not found case.

User Repository

- get_users - admin exclusion, active/inactive filtering, role filtering, pagination.
- count_users - admin exclusion from count.
- switch_status - activate, deactivate, not found.
- admin_delete - success and not found cases.
- update_role - success and not found cases.
- save_user - persists changes and refreshes from database.
- revoke_refresh_token - invalid token handled silently.

Results

- Achieved >75% coverage for core functions.

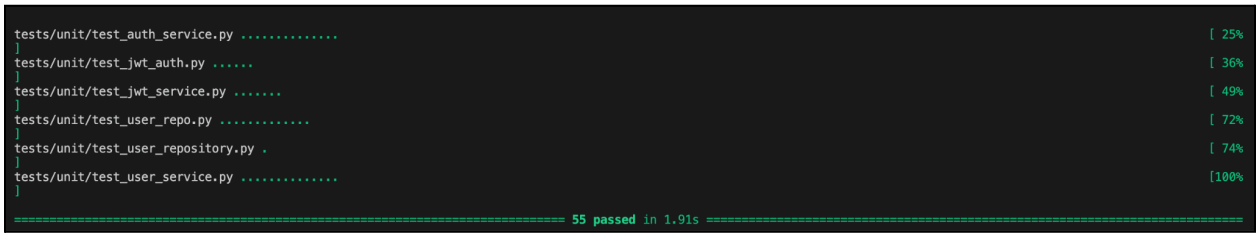

Figure 1 - Unit test results

2. Frontend Component Testing

Overview

Frontend component tests validate UI components in isolation, confirming that each component renders correctly, responds to user interaction, and handles API success and failure states. Tests run against mocked API responses so network behaviour does not affect results.

Approach

Implemented using Vitest and Testing Library. MSW (Mock Service Worker) intercepts API calls and returns controlled responses. Each component is tested for rendering, user interaction, loading states, error states, and navigation behaviour.

Relevance

Component tests catch UI regressions and interaction bugs before they reach E2E or manual testing. They also verify that role-based rendering logic is applied correctly, the wrong nav items appearing for a role, or a form accepting invalid input, are the kinds of bugs that surface here and not in backend tests.

Coverage across components

- Login Page - form rendering, field validation, generic error on 401, password visibility toggle, loading state, successful navigation, and register link.
- Register Page - form rendering, field validation, role selection, email and password validation, loading state, success screen, duplicate account error, and role dropdown options.
- Admin Auth Page - loading and render, empty state, fetch error, accept and reject flows, admin-in-list warning, and API failure handling.
- Navigation - role-based nav item visibility for all four roles, drawer open/close behaviour, and logout state clearing.
- Frontend - Profile Page - profile load, name update validation, password change validation, and successful password change with sign out.
- Frontend - Role Management - loading and render, empty state, fetch error, save button state, successful role change, API failure, and success message timeout.

Results

- Achieved >75% coverage.

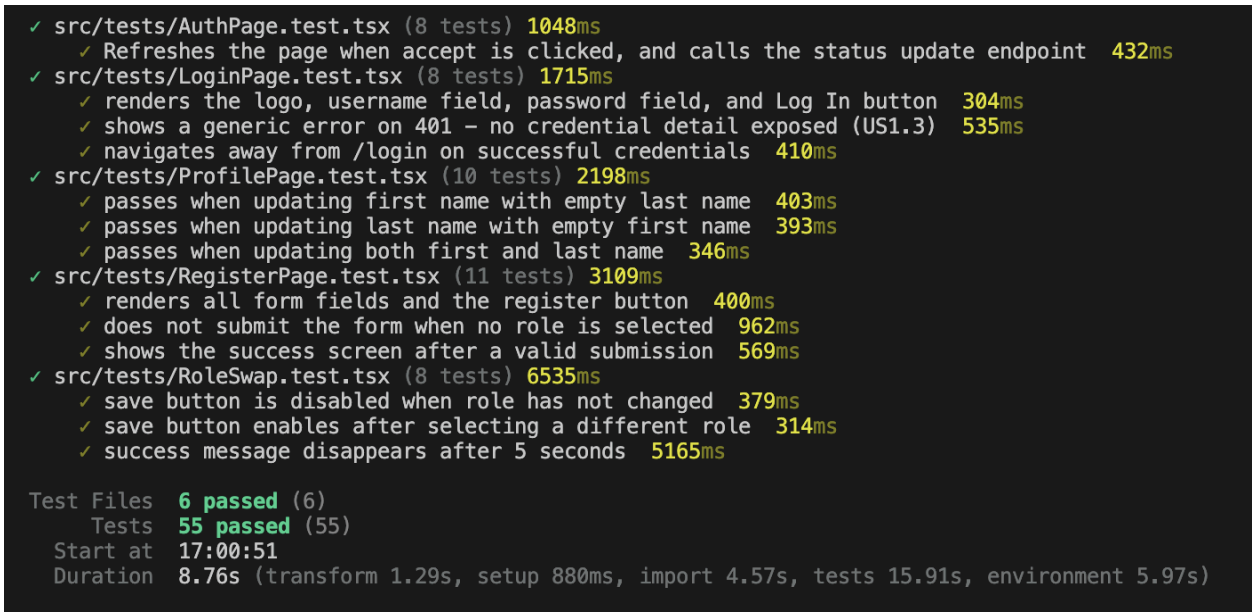

Figure 2 - Frontend test results

3. Integration Testing

Overview

Integration testing validated that modules interact correctly when wired together. Several failure modes cannot surface in unit tests: a correctly implemented service function can still fail if the router applies the wrong RBAC dependency, or if a repository query skips row level filtering for the requesting user. Integration tests are the correct level for catching these.

Approach

All integration tests use the FastAPI TestClient, sending real HTTP requests against the full application stack with a real PostgreSQL + PostGIS test database. No database mocking is applied at this level. The methodology was as follows:

- API-Level Testing - each endpoint tested via HTTP, covering both happy path and failure path scenarios.
- Authentication Enforcement - every protected endpoint tested with missing, invalid, expired, and wrong role tokens. All must return the correct status codes.
- Database Interaction - create, read, update, and delete operations verified against the test database. Each test cleans up after itself.
- RBAC Cross Checks - scenarios such as a non admin attempting to change a user's role, or a request with no token reaching a protected route.
- Test Isolation - test users are prefixed with test_ and removed after each test to prevent state leaking between runs.

Relevance

Integration testing confirmed that the authentication flow works end-to-end, that token rotation and revocation behave correctly under real database conditions, and that role enforcement at the router level matches the service-level expectations established in unit tests. Several edge cases around inactive accounts and enumeration prevention were only fully verifiable at this level.

Coverage

Auth Endpoints

- POST /v1/auth/register - happy path and validation (duplicate email, duplicate username, invalid fields, admin role rejection).
- POST /v1/auth/login - valid credentials, wrong password and unknown username returning identical vague responses, inactive account handling, malformed requests, and security headers.
- POST /v1/auth/refresh - token rotation, old token rejection, and invalid token handling.
- POST /v1/auth/logout - token revocation, idempotent logout, and malformed token handling.

User Endpoints

- GET /v1/users/me - profile retrieval for authenticated user.
- PATCH /v1/users/me - name updates, input validation, password change with revocation of existing sessions, and authentication enforcement.
- PATCH /v1/users/{user_id}/role - role change by admin, rejection of invalid roles, RBAC enforcement for non admin users.

Results

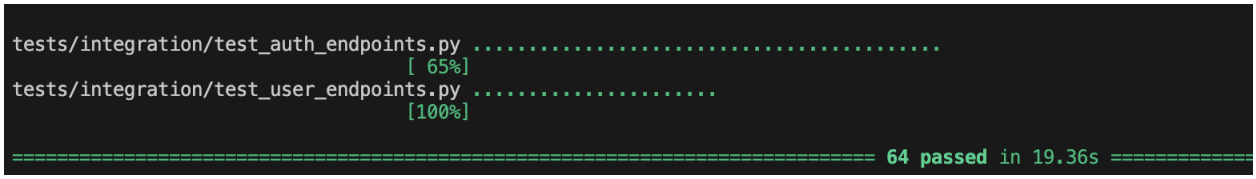

Figure 3 - Integration test results

4. End-to-End Testing

Overview

End-to-End testing simulates full user workflows from login through to task completion, running against a deployed test environment. Cypress is used for browser automation.

Approach

- Workflows mirror typical user journeys for each role.
- Happy path, alternative, and failure paths are covered.
- Offline behaviour is simulated by intercepting network requests via Cypress.

Relevance

E2E tests confirm that the full user journey across the frontend, API, background workers, and database works as intended.

Coverage

Results